A Low-Cost 2D-LiDAR Build for AutoSDV

Cheaper sensing + localization, unchanged Autoware planning

NEWSLabNTU AutoSDV · design report · 2026-07-21

Executive Summary

Goal: a cheaper build variant of AutoSDV — not a different robot. AutoSDV is a research platform built on Autoware, and the 3D LiDAR is its single largest sensor cost. This variant swaps that 3D LiDAR for an inexpensive **2D LiDAR** and keeps **everything that makes AutoSDV a research platform** — Autoware's planning, control, behaviors, and vehicle interface stay exactly as they are.

A 2D LiDAR cannot feed Autoware's 3D localization (NDT) or 3D perception (CenterPoint), so those two modules are replaced/dropped. For localization we borrow a **proven, MIT-licensed** 2D stack from f1tenth — `particle_filter` (Monte-Carlo localization on a 2D occupancy grid). Its pose is bridged into Autoware's existing `ekf_localizer`, so **planning and control see the same `kinematic_state` they always have**.

The change is deliberately **shallow**: it touches only the sensing and localization layers. One small new node (a pose-covariance relay) is the entire custom code footprint. The costs paid for the lower price are (a) authoring a Lanelet2 map as usual plus a 2D occupancy grid, and (b) loss of 3D object perception — acceptable for a low-speed, low-cost research build.

Legend Autoware — unchanged · Borrowed (f1tenth) · New glue · Offline map · Dropped

Motivation — Why a 2D Variant

The LiDAR dominates the AutoSDV sensor bill. A 2D LiDAR is roughly an order of magnitude cheaper than the 3D units AutoSDV ships, while the rest of the sensor suite (IMU, USB camera, GNSS) is untouched. Indicative list prices (vary by vendor and region — treat as ballpark, not a quote):

Class	Example units	Indicative cost
3D LiDAR — mechanical	Velodyne VLP-32C	$~10–30 k
3D LiDAR — solid state	Robin-W, Blickfeld Cube1	$~1–5 k
2D LiDAR — industrial	Hokuyo UST-10LX, SICK TiM	$~1–2 k
2D LiDAR — hobby	Slamtec RPLIDAR	$~0.1–0.6 k

The design constraint is what separates this from a toy conversion: **keep the research value**. AutoSDV's worth is its Autoware planning/control stack and the experiments it enables. So the variant lowers cost **only at the sensor + localization layer** and preserves the full Autoware driving pipeline above it.

What Changes vs Stock AutoSDV

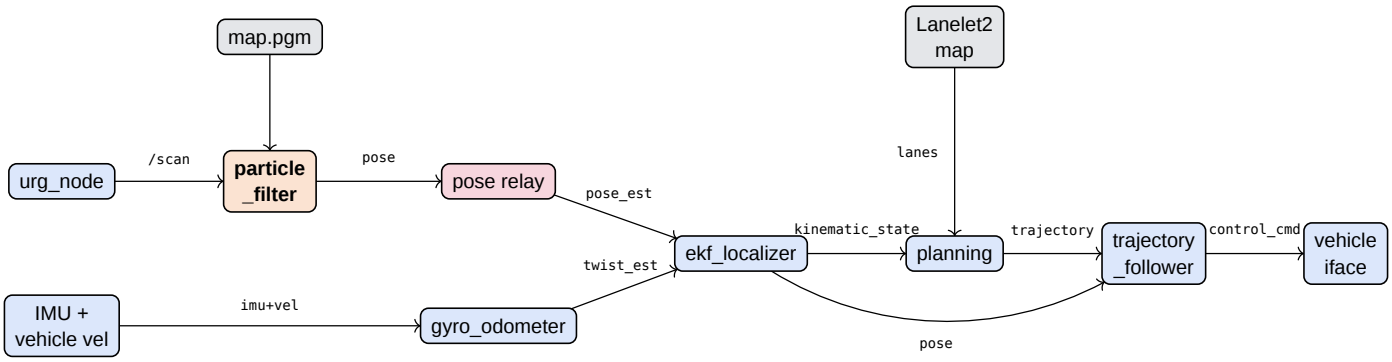
Layer	Stock AutoSDV	2D-LiDAR variant	Status
LiDAR	3D (VLP-32C / Robin-W / Cube1)	2D (Hokuyo)	swap
Localization	NDT scan matching (3D)	`particle_filter` (2D MCL)	replace
Geometry map	PCD point cloud	2D occupancy grid	replace
Vector map	Lanelet2	Lanelet2	**unchanged**

Layer	Stock AutoSDV	2D-LiDAR variant	Status
3D perception	CenterPoint	dropped → optional 2D obstacle stop	reduce
Planning	behavior + motion	behavior + motion	**unchanged**
Control	`trajectory_follower`	`trajectory_follower`	**unchanged**
Vehicle interface	AutoSDV interface	AutoSDV interface	**unchanged**
Custom code	—	pose-covariance relay (1 node)	new

Four rows change; the entire planning/control half of the stack — the research surface — is untouched.

Architecture

The 2D front-end (LiDAR + `particle_filter`) replaces the 3D front-end. Its pose is relayed into Autoware's `ekf_localizer`, which fuses it with the existing twist estimate and emits the same `/localization/kinematic_state` that planning and control already consume. The output side is **native Autoware** — no adapter, no message translation at the vehicle.



The Localization Bridge

The one seam that needs code. `particle_filter` produces a 2D pose; Autoware's `ekf_localizer` expects a pose estimate on the exact topic NDT used to publish:

`ekf_localizer` input	Fed by the variant
`…/pose_estimator/pose_with_covariance`	`particle_filter` pose → **pose relay** (stamps covariance, re-frames to `map`)
`…/twist_estimator/twist_with_covariance`	`gyro_odometer` — IMU + vehicle velocity (**already in AutoSDV**)

`ekf_localizer` fuses the two into `/localization/kinematic_state` and the `map→base_link` TF. Downstream, **nothing else in Autoware knows the LiDAR changed**. If `particle_filter` can be configured to publish the right topic and covariance directly, the relay collapses to launch config.

Sensor & Message Shapes

The change is, concretely, a swap of ROS 2 message types at two points: the range sensor (both are consumed differently) and the localization pose (the bridge). Planning/control messages are unchanged.

Sensor Interface

Signal	Stock AutoSDV	2D-LiDAR variant
Range	`sensor_msgs/PointCloud2` — `/sensing/…/points` (3D `x y z intensity`)	`sensor_msgs/LaserScan` — `/scan` (1 plane: `ranges[]`, `angle_*`)
IMU	`sensor_msgs/Imu`	`sensor_msgs/Imu` (unchanged)
Wheel speed	`autoware_vehicle_msgs/VelocityReport` → twist	same (unchanged)
GNSS	`sensor_msgs/NavSatFix` → pose	same (optional, for init)

Only the range sensor changes: `PointCloud2` (3D) → `LaserScan` (2D). Every other sensor interface is identical.

Localization

Aspect	Stock — NDT	Variant — particle_filter
Algorithm	NDT scan matching (3D registration)	Monte-Carlo localization (2D, RangeLibc raycast)
Map input	`PointCloud2` (PCD)	`nav_msgs/OccupancyGrid` (map_server)
Scan input	`PointCloud2` (3D)	`sensor_msgs/LaserScan` (2D)
Motion input	ekf twist (IMU + wheel)	`nav_msgs/Odometry` `/odom`
Pose output	`PoseWithCovarianceStamped` → `ekf_localizer`	`nav_msgs/Odometry` + `PoseStamped` → **relay** → `ekf_localizer`
Fusion	`ekf_localizer` → `kinematic_state`	`ekf_localizer` → `kinematic_state` (**same**)
Init	`pose_initializer` (GNSS / NDT / manual)	`/initialpose` (manual RViz) or GNSS

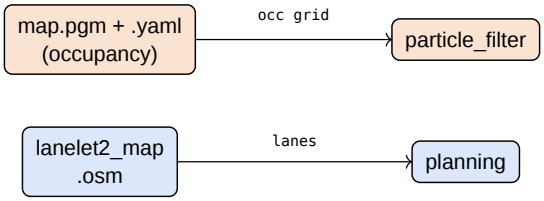
Message Shapes (the two conversions)

Role	Stock AutoSDV msg	Variant msg
Range (sensor swap)	`sensor_msgs/PointCloud2` `{header, height, width, fields[x,y,z,intensity], data[]}`	`sensor_msgs/LaserScan` `{header, angle_min, angle_max, angle_increment, range_min/max, ranges[], intensities[]}`
Ego pose (bridge)	`geometry_msgs/PoseWithCovarianceStamped` `{header, pose.pose{position, orientation}, pose.covariance[36]}`	`nav_msgs/Odometry` (from PF) `{header, child_frame_id, pose.pose{…}, pose.covariance[36], twist.twist{…}}` → relay reshapes to the Autoware msg on the left

The **pose relay** exists to reshape the ego-pose row: `particle_filter` `Odometry` / `PoseStamped` → `PoseWithCovarianceStamped`, populating a realistic covariance so `ekf_localizer` weights it correctly.

Maps

The variant needs **two** offline maps. The Lanelet2 vector map is **the same artifact stock AutoSDV already requires** for planning — no new burden. The only new map is the 2D occupancy grid that replaces the PCD point cloud (cheaper to build: one teleop lap + `slam_toolbox`, versus a survey-grade 3D cloud).



Dimension	Stock geometry map (PCD)	Variant geometry map (occupancy)
Format	`.pcd` — PCD v0.7 binary, `x y z [rgb]`	`.pgm` — grayscale occupancy image + `.yaml`
Frame	Georeferenced WGS84 / MGRS	Local metric; grid origin in `.yaml`
Built with	3D LiDAR SLAM (survey-grade)	2D SLAM (`slam_toolbox`), one teleop lap
Typical size	10s of MB (27 MB sample)	~100 KB
Effort	**high**	**low**

Vector map (Lanelet2) is unchanged. `.osm` OSM-XML with lanes, stop lines, and regulatory elements — authored in Vector Map Builder / JOSM exactly as for stock AutoSDV. Align its origin to the occupancy grid frame.

Preferred grid source for existing sites: slice the PCD. Crop the existing 3D point-cloud map to a z-band at the 2D LiDAR mounting height (relative to local ground), project to XY, rasterize at 0.05 m → `map.pgm` + `.yaml` (`pcd-to-pgm.py`). Because the PCD and the Lanelet2 map share one georeferenced frame, the grid is **aligned to the vector map by construction** — no re-mapping, no manual alignment. `slam_toolbox` remains the path for new sites without a PCD. Caveats: sloped sites need ground-relative slicing; survey-day clutter becomes phantom occupancy (erase manually if it disturbs MCL).

Perception Trade-off

With no 3D LiDAR there is no CenterPoint object detection. Planning runs on an **empty or scan-derived** object list. Options: (a) **static-environment** mode — no dynamic objects, planner follows the lane; (b) a **2D obstacle stop** — cluster `/scan` or the occupancy grid into blockages that trigger Autoware's stop behavior. Dynamic overtaking/avoidance is **not** available with a 2D-only sensor. This is the main capability cost of the cheaper build.

Phased Integration

Phases 0–2 are **offline preparation** with standalone checks — no vehicle, no full launch. Phases 3–7 run on **rosbag replay** (Autoware's official replay simulation assets or an AutoSDV recording), so the entire pipeline is bench-tested before hardware. Phase 8 is the only on-vehicle step.

Phase	Work	Verify (standalone where possible)
0	**Vendor & build** — add Roboracer `particle_filter` + `range_libc` as submodules (upstream URLs; fork to NEWSLabNTU only if patched); `colcon build`	Build succeeds

Phase	Work	Verify (standalone where possible)
1	**Map preparation** — `pcd-to-pgm.py`: z-band slice of the site PCD → `map.pgm` + `.yaml` (frame-aligned with Lanelet2 by construction). Alt for new sites: teleop + `slam_toolbox`	**Standalone check:** `map_server` loads the grid; RViz overlay of grid vs Lanelet2 lanes aligns; origin/resolution sane
2	**Rosbag preparation** — fetch Autoware `sample-map-rosbag` + `sample-rosbag` (or COSS bag via `just download-data`); synthesize `/scan` from the 3D pointcloud with `pointcloud_to_laserscan` (z-band at virtual mount height)	**Standalone check:** replay renders `/scan` in RViz at expected rate; scan geometry matches walls in the grid
3	**Port MCL** — `particle_filter` on the prepared grid, fed by replayed `/scan` + `/odom`; run stock NDT on the same bag in parallel as ground truth	PF pose vs NDT pose error bounded (quantitative, not eyeball)
4	**Localization bridge** — pose relay → `pose_estimator/pose_with_covariance`; enable `ekf_localizer` + `gyro_odometer`	`/localization/kinematic_state` + `map→base_link` TF valid; still tracks NDT reference
5	**Perception wiring** — `laserscan_to_pointcloud` → `obstacle_segmentation/pointcloud`; standalone `LaserscanBasedOccupancyGridMapNode` from real `/scan` → `occupancy_grid_map/map`	**Standalone check:** both topics at rate; live grid renders; planted obstacle in replay appears in cloud + grid
6	**Planning/control port** — forked `autosdv_planning/control_component` launches (config root → `autosdv_launch`); `2dlidar` preset (module policy); param overrides (costmap height gate, collision_detector pointcloud, velocity-limiter grid source)	Planner boots with `empty_objects_publisher`; no module blocks readiness; trajectory published on replay
7	**Replay end-to-end** — route/goal on replayed run; obstacle-stop drill — **no actuation**	Trajectory follows lanes; `obstacle_stop` inserts stop at planted blockage
8	**Vehicle bring-up** — 2D LiDAR driver + static TF `base_link→laser`; native `control_cmd` → vehicle interface; low-speed closed loop	Live `/scan` matches replay behavior; car follows the lane; stops at obstacle

Only phase 8 needs the vehicle and the physical 2D LiDAR. NDT-as-reference in phases 3–4 turns localization sign-off into a measured comparison rather than an RViz impression.

Phase 3 Outcome — Localization Verdict

The MCL localization path failed its Phase 3 gate because the measurement model was mis-specified — and Phase 3e found and fixed that mis-specification. Phase 3d instrumentation showed the beam model's likelihood maximum sitting 29–30 m from the true pose (47–50 nat gap) even while the filter tracked to 0.4 m, and traced it to two implementation defects: an unnormalised `z_short` mixture component (the configured 75% hit weight degraded to 6.8–25.4% depending on range) and no-return beams outscoring perfectly matching ones (30% of beams non-finite in practice, each worth 1.87× a matching beam). Phase 3e normalised `p_short`, dropped non-finite beams from the product, and gated the correction step to run once per scan instead of twice.

The Phase 3 accuracy gate now passes. End-to-end, 5 replay seeds on the official Autoware sample site each meet mean < 1.0 m / p95 < 2.5 m / yaw < 0.2 rad: mean translational error 0.79 m (range 0.76–0.81 m across seeds), p95 2.02 m (range 1.95–2.04 m), mean |yaw| error 0.026 rad (range 0.026–0.028 rad) — 5/5 seeds passing, against 0/5 for the unfixed upstream model (15.9–36.0 m mean error). The offline sensor-model gap also collapsed, from a 51.9 nat median GT-to-argmax gap (30 m offset) to 6.2 nats (0.2 m offset).

Consequence for this design: 2D MCL is a viable localization source for this build — the localization column of the architecture can proceed on the vendored `particle_filter`, not just on NDT / `cuda_ndt` as a fallback. Caveats: this is measured on a single site (the official Autoware sample bag) with one map, not yet cross-validated on COSS or a live run; the three fixes live in the NEWSLabNTU fork (`particle_filter`, branch `autosdv`), not upstream; and one fix has an **interface consequence** — correcting once per scan instead of once per odometry message halves the pose and TF publish rate (≈10 Hz instead of ≈20 Hz), because the filter has no predict-only publish path. The `ekf_localizer` bridge described above must be checked against that rate before it is relied on.

Open gap — initialization. The measured runs seed the filter with the first NDT ground-truth pose, an oracle unavailable on a real vehicle, so the figures above characterise **tracking given a correct starting pose**, not global localization. NDT itself initialises properly from `gnss_poser` through `autoware_pose_initializer`; the 2D-MCL variant must reuse that same path, which is the first Phase 4 task. Details: `docs/research/localization/2d_mcl_algorithm.md`, `docs/reports/2dlidar-phase3e-model-fixes.md`.

Open Risks

Risks below are the original design-time list. Those that Phase 3 has now **resolved into measured findings** are marked; the rest still stand.

Risk	Mitigation
Covariance realism — `ekf_localizer` trusts the relay's reported covariance; bad values destabilize the fusion.	Tune PF covariance; validate `kinematic_state` against RViz ground truth before enabling planning.
2D perception gap — planner is blind to obstacles outside the scan plane.	Ship a 2D obstacle-stop node; keep speeds low; document the limitation.
Pose initialization — PF needs a 2D initial pose; no NDT auto-init.	Manual RViz 2D-pose-estimate, or GNSS via `pose_initializer` when available.
Odom quality — PF's motion model leans on `/odom`; poor wheel odometry degrades MCL. **Resolved (Phase 3): not the bottleneck.**	Measured: wheel+IMU dead reckoning holds 0.79 m mean over a 28 s drive once the IMU yaw-rate sign is corrected. The failure lies in the measurement model, not the motion model.
Measurement-model mis-specification (found Phase 3d) — the beam model's likelihood maximum was 29–30 m from truth (47–50 nat gap); unnormalised `z_short` and no-return beams outscoring matches. **Resolved (Phase 3e): gate passes 5/5 seeds.**	Fixed: normalised `p_short`, dropped non-finite beams, gated correction to run once per scan (NEWSLabNTU fork, branch `autosdv`). Measured: offline gap 51.9→6.2 nats median; end-to-end 0.79 m mean / 2.02 m p95 / 0.026 rad yaw over 5 seeds (all pass), vs.

Risk	Mitigation
	0/5 for upstream. Single-site result — not yet cross-validated on COSS or live hardware.
Planner fit — Autoware defaults tuned for full-size vehicles.	Set vehicle footprint, min turning radius, and velocity limits for the platform.

Rejected Alternative — Full f1tenth Racing Stack

An alternative would replace the **entire** driving loop with f1tenth's racing stack (raceline planner + pure-pursuit control), using Autoware only as a sensor/vehicle shell and a `/drive` → `Control` adapter at the output.

Rejected. AutoSDV is a research platform, not a race car. Swapping in the f1tenth planner/controller would discard exactly what gives AutoSDV its value — Autoware's behavior/motion planning, safety framework, and the experiments built on them — turning it into a roboracer-class vehicle. The goal here is a **cheaper AutoSDV**, so the change is kept shallow: only sensing and localization move to the low-cost 2D stack; planning and control remain Autoware.

Generated as a design report · sources: `~/repos/AutoSDV`, f1tenth `particle_filter`, Autoware localization launch